CSC490 Assignment A4 - RL-ing Nanochat

Yibing Ju (1009382829), Cynthia Zhou (1009028918), Jonathan Chen (1008849310), Mark Noge (1008992299)

1 Part One: GRPO and RL Review

Karpathy's nanochat RL implementation deviates from standard GRPO in four key ways. First, the KL divergence penalty and reference model are removed entirely. At nanochat's scale, the memory cost of a frozen reference model is unjustified and catastrophic policy drift is unlikely. Second, the PPO importance sampling ratio and clipping are unnecessary and dropped, since samples are regenerated fresh from the policy every step. Third, advantage normalisation omits the standard deviation term, which avoids division-by-zero instability when all sampled rewards are identical - this can be a common occurrence early in GSM8k training when all outputs get the question wrong. Finally, loss normalisation is done at the token level rather than the sequence level, ensuring longer math reasoning chains are not implicitly penalised relative to shorter responses. Together, these changes utilize the bare essential ideas of GRPO while avoiding failure modes specific to small-scale math reasoning training.

2 Part Two: SFT & Midtraining

Pretrained Model

Checkpoint: `nanochat_d26_diff_attn` (from A3 Part 4)
Architecture: d=26, ~830M params, differential attention, RoPE, GQA, QK-norm, ReLU²
Pretrain metrics: val_bpb=0.7576, CORE=0.2405

Baseline SFT Mixture (Original chat_sft.py)

Dataset	Size	Purpose
SmolTalk	460K	General conversations
Identity conversations	1K × 2 epochs	Personality

MMLU auxiliary train	100K × 3 epochs	Multiple choice reasoning
GSM8K train	8K × 4 epochs	Math + tool use
SimpleSpelling	200K	Spelling
SpellingBee	80K	Letter counting
Total	**~1.17M rows**	

Additional Math Datasets

We select three math SFT datasets to test as augmentations, chosen based on our research in `dev/math_datasets_research.md`. Selection criteria: (1) directly targets GSM8K-style problems, (2) plain-text CoT format compatible with nanochat, (3) manageable size (200K–600K) to avoid catastrophic forgetting at 830M params.

MetaMathQA (Yu et al., arXiv:2309.12284)

- **HuggingFace**: `meta-math/MetaMathQA`
- **Size**: 395K rows
- **License**: MIT
- **Description**: Augmented math Q&A bootstrapped from GSM8K and MATH train sets via GPT-3.5. Uses question rephrasing, answer augmentation, self-verification, and backward reasoning (FOBAR).
- **Justification**: Directly derived from GSM8K questions with diverse rephrasings — the most targeted augmentation for our GSM8K goal. MetaMath-7B achieved 66.5% GSM8K (vs 14.6% base LLaMA-2-7B).

Orca-Math (Mitra et al., arXiv:2402.14830)

- **HuggingFace**: `microsoft/orca-math-word-problems-200k`
- **Size**: 200K rows
- **License**: MIT

- **Description**: Synthetic grade-school math word problems created via multi-agent GPT-4 Turbo pipeline (agent generates, teacher verifies/refines).
- **Justification**: Specifically designed for small language models on grade-school math. A 7B model trained *only* on this 200K dataset surpassed LLaMA-2-70B and ChatGPT-3.5 on GSM8K. Smallest dataset in our set — tests whether quality > quantity at 830M params.

DART-Math-Hard (Tong et al., arXiv:2407.13690, NeurIPS 2024)

- **HuggingFace**: `hkust-nlp/dart-math-hard`
- **Size**: 585K rows
- **License**: MIT
- **Description**: Created via Difficulty-Aware Rejection Tuning (DARS), which allocates more generation attempts to harder problems from GSM8K and MATH train sets.
- **Justification**: Oversamples problems that models struggle with — addresses the failure mode where easy problems dominate training. Largest dataset in our set — tests whether hard-problem oversampling helps at 830M scale.

Experimental Runs

Run	Tag	Mixture	Total Rows (approx)
1	`sft_baseline`	Original SFT	~1.17M
2	`sft_metamathqa`	Original + MetaMathQA	~1.57M
3	`sft_orcamath`	Original + Orca-Math	~1.37M
4	`sft_dartmath`	Original + DART-Math-Hard	~1.75M
5	`sft_combo`	Original + MetaMathQA + DART-Math-Hard	~1.75M

All runs use:

- Same pretrained checkpoint: `nanochat_d26_diff_attn`
- Same training config: `--device-batch-size=8` (d=26 OOM at 16 during SFT)
- Same optimizer settings inherited from pretrain checkpoint
- W&B project: `nanochat-part2`

Run 5 combines MetaMathQA (the only dataset that improved GSM8K) with DART-Math-Hard (the strongest non-GSM8K gains: best ARC-Easy and MMLU).

Results

ChatCORE Evaluation

Run	Tag	ARC-Easy	ARC-Challenge	MMLU	GSM8K	HumanEval	SpellingBee
1	sft_baseline	56%	42%	34%	4%	2%	100%
2	sft_metamathqa	62%	40%	34%	**18%**	4%	100%
3	sft_orcamath	60%	38%	40%	0%	2%	100%
4	sft_dartmath	64%	28%	**44%**	0%	4%	100%
5	sft_combo	52%	**46%**	38%	**20%**	**6%**	100%

All evaluations use `--max-problems=50` with cache-free generation (differential attention).

Training Metrics

Run	Tag	val_bpb	Total Rows

1	sft_baseline	0.3679	~1.17M
2	sft_metamathqa	0.3677	~1.57M
3	sft_orcamath	0.3671	~1.37M
4	sft_dartmath	0.3669	~1.75M
5	sft_combo	0.3671	~1.75M

All runs converge to nearly identical val_bpb (0.3669–0.3679), indicating that the additional math datasets do not meaningfully change overall SFT loss — the performance differences between runs are only visible in downstream task evaluations.

Analysis (Runs 1–4)

1. Does any single math dataset meaningfully improve GSM8K over the baseline?

Yes, but only MetaMathQA. It improved GSM8K from 4% to 18% (+14pp). Orca-Math and DART-Math-Hard both scored 0%, *worse* than baseline. This suggests that MetaMathQA's direct bootstrapping from GSM8K train questions (via rephrasing, FOBAR, etc.) transfers far more effectively than general math problem sets at the 830M scale.

2. Does adding math data hurt other tasks (catastrophic forgetting)?

Minimal forgetting observed. All math-augmented runs maintained or improved ARC-Easy (56% → 60–64%) and MMLU (34% → 34–44%). ARC-Challenge dropped for DART-Math (42% → 28%), suggesting some trade-off with the largest augmentation. SpellingBee remained at 100% across all runs. HumanEval stayed near the floor (2–4%) regardless.

3. Does quality (Orca-Math, 200K) or quantity (DART-Math, 585K) matter more?

Neither helped GSM8K at 830M params. Orca-Math (quality-focused, 200K) and DART-Math (hard-problem oversampling, 585K) both scored 0% on GSM8K. The key differentiator appears to be *domain alignment*: MetaMathQA is bootstrapped directly from GSM8K questions, while

the others generate novel problems. At a small model scale, direct augmentation of the target distribution matters more than general math coverage.

4. Non-GSM8K benefits

DART-Math showed the strongest gains on MMLU (+10pp) and ARC-Easy (+8pp), suggesting its hard-problem diversity helps general reasoning even if not GSM8K specifically. This motivated its selection for the combo run alongside MetaMathQA.

Combo Run Rationale

We combine MetaMathQA (the only GSM8K improver) with DART-Math-Hard (the strongest general reasoning gains). The hypothesis: MetaMathQA provides GSM8K-targeted signal while DART-Math provides broad math reasoning. The risk: the larger combined mixture (~1.75M rows) may dilute MetaMathQA's GSM8K benefit.

Analysis (Run 5 — Combo)

5. Does combining datasets preserve MetaMathQA's GSM8K gains?

Yes — and slightly improves them. The combo achieved 20% GSM8K, edging out MetaMathQA alone (18%). DART-Math's additional math reasoning data did not dilute the GSM8K signal from MetaMathQA.

6. Does the combo achieve "best of both worlds" or average out?

Mostly best of both worlds, with one trade-off. The combo achieved the best scores across GSM8K (20%), ARC-Challenge (46%), and HumanEval (6%). MMLU landed between MetaMathQA (34%) and DART-Math (44%) at 38%, which is a mild averaging effect. However, ARC-Easy unexpectedly dropped to 52% (below even the baseline's 56%), suggesting some interference in easy factual recall.

Overall Summary

The combo run (`sft_combo = baseline + MetaMathQA + DART-Math-Hard`) is our best overall model:

- **GSM8K +16pp** over baseline (4% → 20%) — the primary goal
- **ARC-Challenge +4pp** over baseline (42% → 46%)
- **HumanEval +4pp** over baseline (2% → 6%)
- **MMLU +4pp** over baseline (34% → 38%)
- **ARC-Easy -4pp** vs baseline (56% → 52%) — minor regression

- **SpellingBee** unchanged at 100%

At 830M params with only 1 epoch of SFT, a +16pp GSM8K improvement from data augmentation alone (no RL) is a meaningful result. The finding that only MetaMathQA (directly bootstrapped from GSM8K) improved GSM8K, while general math datasets did not, highlights the importance of domain-specific augmentation at a small model scale.

The code changes for this part are available in the following branch:

https://github.com/jonathanjschen913/nanochat/tree/a4_part2

A pull request for these changes can also be viewed here:

https://github.com/jonathanjschen913/nanochat/pull/4

3 Part Three: Replicating our RL run with additional analysis

Setup

Starting Checkpoint

We use `sft_combo` — the best performing model from Part 2 (baseline + MetaMathQA + DART-Math-Hard), which is a significantly stronger starting point than Karpathy's vanilla SFT:

Checkpoint	GSM8K before RL
Karpathy vanilla SFT	4.55%
Our `sft_combo`	**20.0%**

The gap is expected: our SFT included 395K MetaMathQA examples bootstrapped directly from

GSM8K train questions, and 585K DART-Math-Hard problems with hard-problem oversampling.

RL Configuration

All hyperparameters match Karpathy's defaults in `scripts/chat_rl.py`. Mid-training evaluation was disabled (`--eval-every=999`) to reduce compute cost (~$35 savings). After training, `stage_collect_completions` runs a single combined stage that collects greedy completions on all 1,319 test problems (for error analysis), reports pass@1 accuracy, and runs pass@8 sampling eval — replacing the previously separate `stage_eval` and `stage_collect_completions` stages.

Parameter	Value
Epochs	1 (467 steps)
Examples per step	16
Samples per example	16
Max new tokens	256
Temperature	1.0
Top-k	50
Device batch size	8
GPU	8×H100
Steps completed	**466 / 467 (full epoch)**

Differential Attention KV Cache Fix

Our model uses differential attention (Ye et al., ICLR 2025). The original nanochat code raised `NotImplementedError` when KV cache was used with diff_attn, forcing an $O(T^2)$ cache-free fallback that made each RL step ~10–20× slower.

We implemented KV cache support for differential attention in `nanochat/gpt.py`. The key insight: `self.n_kv_head` is halved in diff_attn (13 instead of 26), so `2 × self.n_kv_head == config.n_kv_head` — the existing `KVCache` is already the right size. We partition the cache heads:

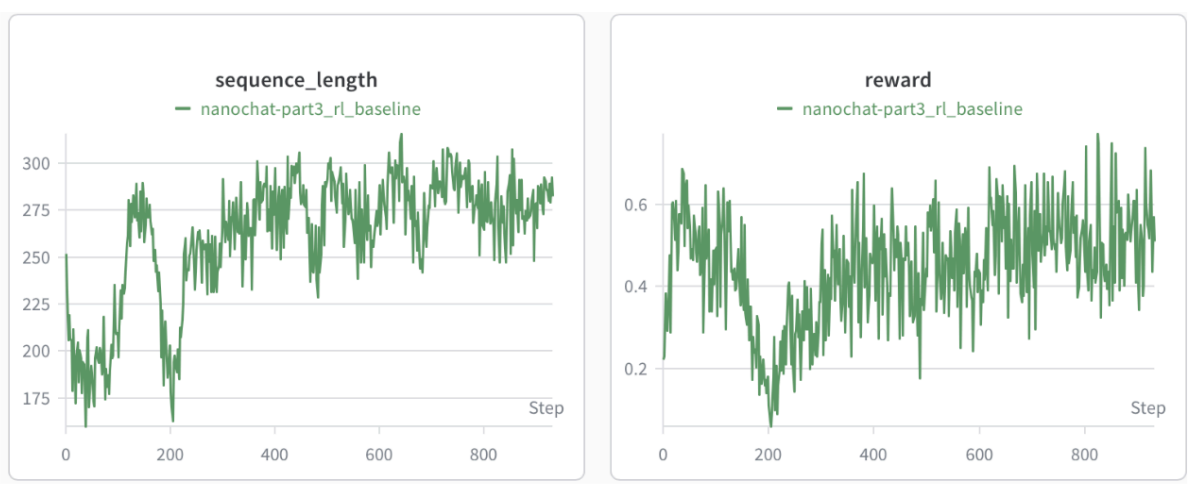
- **First `n_kv_head` slots** → k1 / v1
- **Last `n_kv_head` slots** → k2 / v2

During decode, new k1/k2/v1/v2 tokens are written into their slots and attention is computed over the full cached sequence via `flash_attn_func`. No changes to `KVCache` or `Engine` were needed. Output correctness verified: max logit diff between cached and cache-free = **0.000000**.

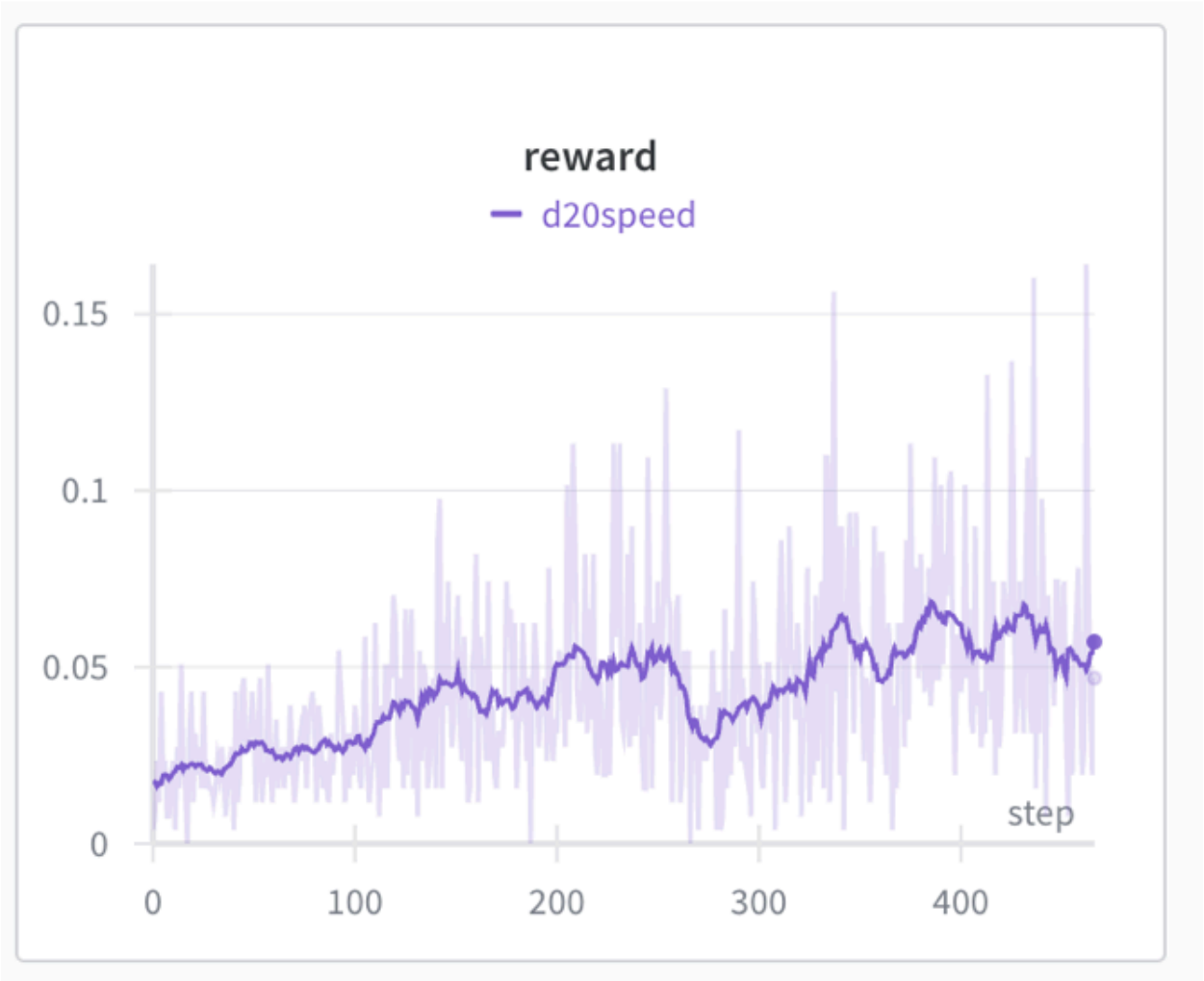
Results

Reward Curve

Our RL reward curve (W&B, `nanochat-rl` project):



Karpathy's RL results (from nanochat discussion #1):



Karpathy reports reward increases steadily over ~1.5 hours. The discussion reports the following progression across training stages:

Stage	GSM8K
After mid-training	2.50%
After SFT	4.55%

After RL	**7.58%**

These are the reference numbers we compare against throughout this writeup. Exact reward curve axis values are not published — only the qualitative trend and final metrics above.

Both runs show the same qualitative pattern: reward climbs consistently over training. Our run starts at a higher reward baseline (~0.22 at step 0, consistent with our stronger SFT) and rises to ~0.59 by step 58, continuing upward through the full epoch.

Evaluation

Metric	Karpathy (vanilla SFT → RL)	Ours (`sft_combo` → RL)
GSM8K before RL (pass@1 greedy)	4.55%	**20.0%**
GSM8K after RL (pass@1 greedy)	7.58%	**25.55%**
RL absolute improvement	+3.03pp	**+5.55pp**
GSM8K after RL (pass@8 sampled)	N/A	**35.75%**
Pass@1 at step 0	4.55%	**11.5%**
Pass@8 at step 0	N/A	**33.25%**

Pass@8 >> Pass@1	Yes	**Yes** (35.75% vs 25.55%)
Training time	~1.5 hrs	~4 hrs

Commentary on Differences

1. Higher absolute scores.
Our model reaches 25.55% vs Karpathy's 7.58% — a 3.4× improvement. This is directly attributable to the stronger SFT starting point (20% vs 4.55%) from MetaMathQA and DART-Math augmentation in Part 2.

2. Smaller relative gain from RL.
Karpathy: +67% relative (4.55% → 7.58%). Ours: +28% relative (20% → 25.55%). The diminishing relative return suggests we are closer to the ceiling of what one epoch of binary-reward RL can extract — the model has already learned much of the GSM8K distribution from SFT.

3. Pass@8 >> Pass@1 in both runs.
Our 10pp gap (35.75% − 25.55%) mirrors Karpathy's observation that the model "knows" the right approach under sampling but cannot reproduce it consistently at greedy decoding. This gap represents untapped capability that more RL epochs or shaped rewards (Part 4) can close.

4. Slower training.
Our ~4 hrs vs Karpathy's ~1.5 hrs is due to differential attention requiring 2× attention calls per forward pass (dual Q/K/V decomposition), even with KV cache. The gradient signal is identical.

Error Analysis

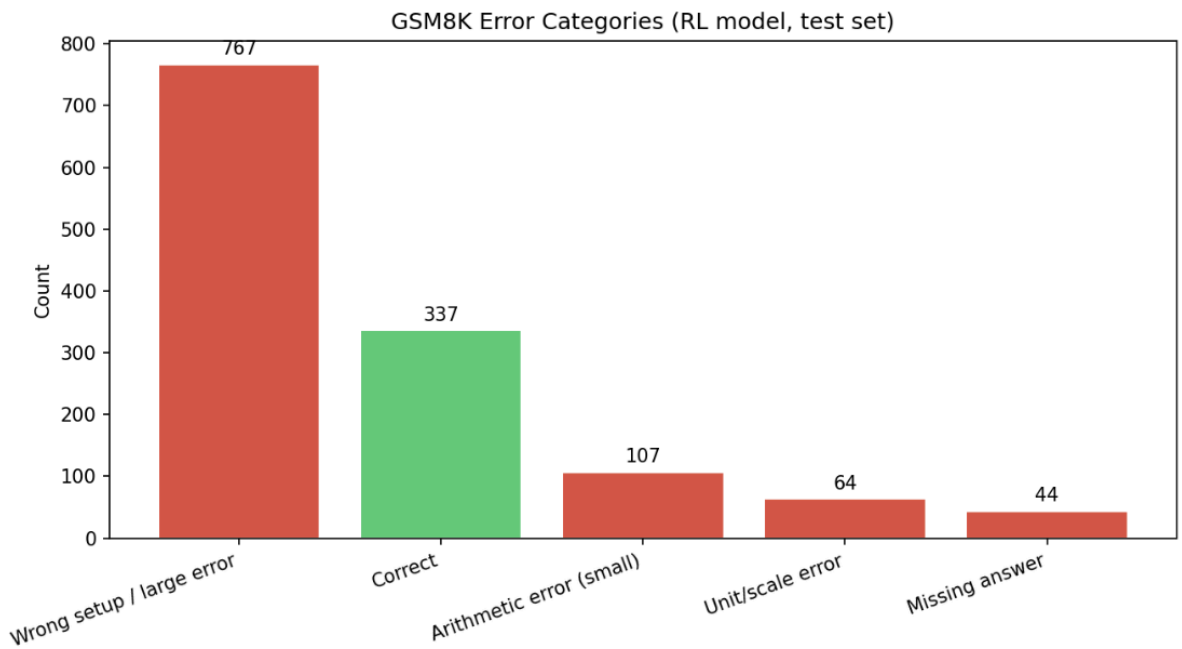
Methodology

We generated greedy (temperature=0) completions for all **1,319 GSM8K test problems** using the RL-trained model, saving each with its correctness label to `dev/part3_completions.jsonl`. Incorrect responses were categorized by pattern matching on the completion text.

Overall Accuracy

337 / 1319 = 25.5% correct (pass@1 greedy, full test set).

Error Categories



Category	Count	% of Total
✅ Correct	337	25.5%
❌ Wrong setup / large error	767	58.2%
❌ Arithmetic error (small)	107	8.1%

❌ Unit / scale error	64	4.9%
❌ Missing answer (`####` absent)	44	3.3%

Key finding: The dominant failure mode is **wrong problem setup (58.2%)** — the model misunderstands the structure of the problem entirely, not just the arithmetic. This is more fundamental than arithmetic errors (8.1%) and directly informs our Part 4 reward design.

Sample completions by error type:

Wrong setup:

Q: "An airport has 2 planes… the first goes to Greece for three-quarters of its flights…"
Gold: `11` | Model sets up the flight distribution incorrectly and arrives at a different total

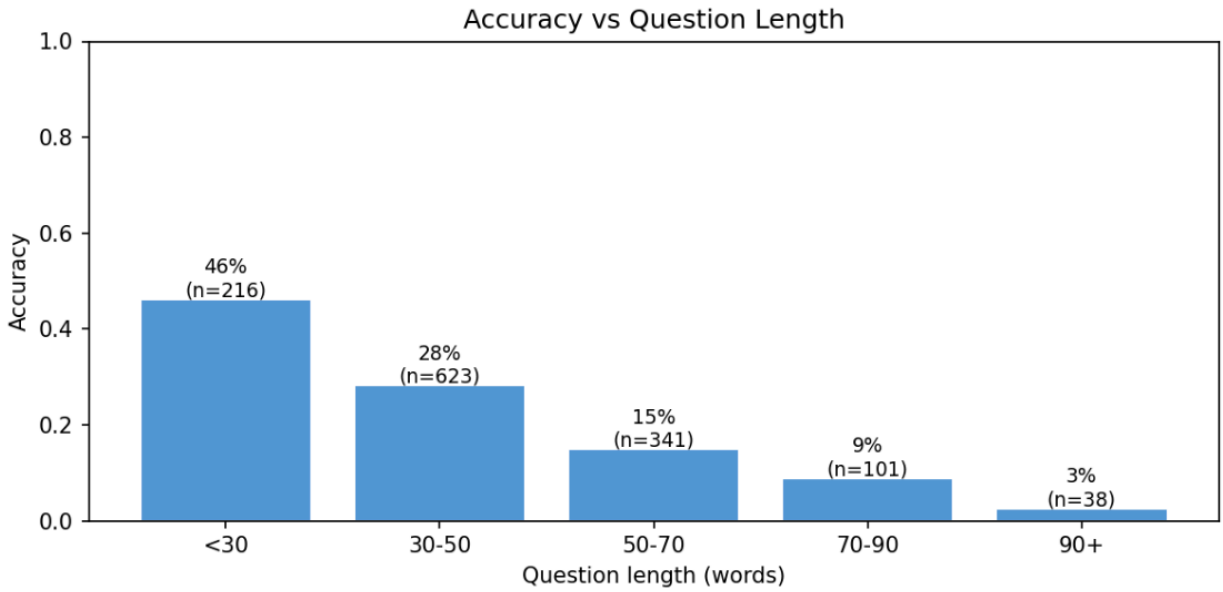
Arithmetic error (small):

Q: "Tyrion changes his face mask two times every time he goes out. Goes out 3 times a day…"
Gold: `12` (daily) | Model: `84` (multiplies by 7 days — correct weekly but wrong scope)

Missing answer:

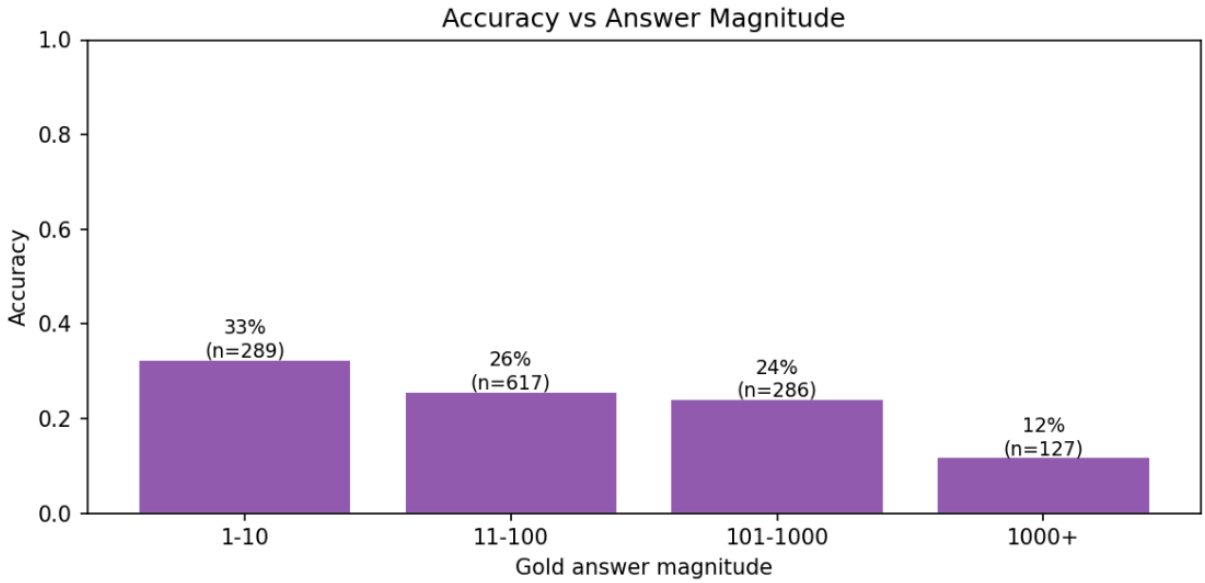
Q: "James loves swimming… swims 60% of the distance…"
Gold: `17` | Model reasons correctly but truncates before generating `####`

Accuracy vs Problem Length



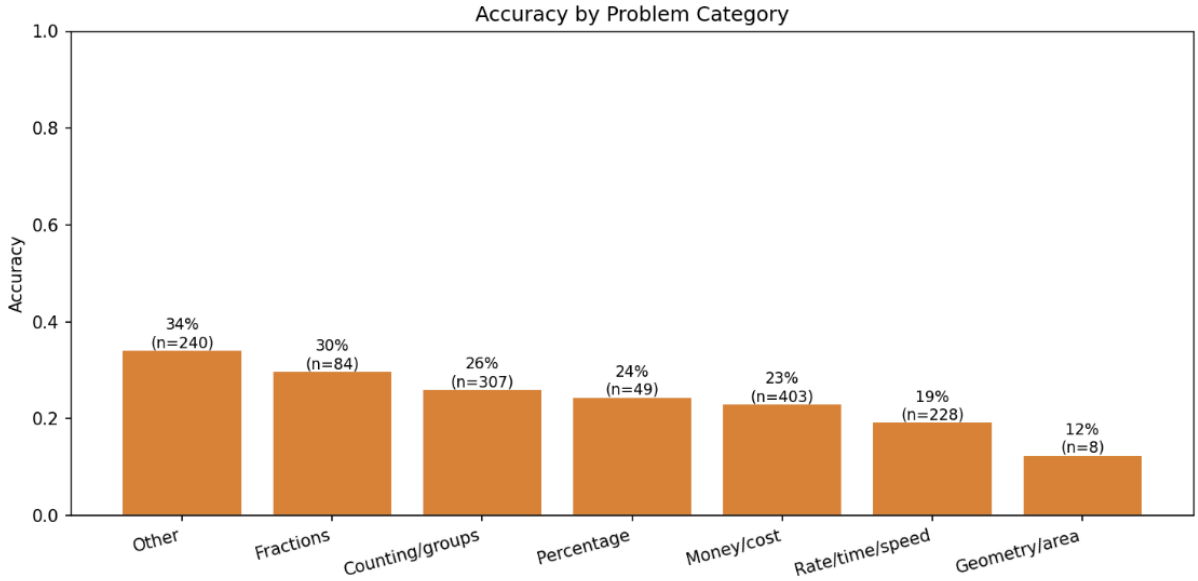
Accuracy decreases as question word count increases. Short problems (<30 words) are typically single-step; longer problems (70+ words) involve tracking multiple characters and conditions across several arithmetic operations, compounding the chance of a setup error.

Accuracy vs Answer Magnitude



Problems with small final answers (1–10) are easier than those requiring large answers (1000+). Large answers require more intermediate multiplication/division steps, increasing the chance of an arithmetic error propagating to a large final discrepancy.

Accuracy by Problem Category



The model performs best on money/cost and rate/time/speed problems — these have formulaic structure that aligns closely with the MetaMathQA and GSM8K SFT training distribution. Percentage and geometry problems show lower accuracy, likely because they require multi-step reasoning with less stereotyped structure.

Commentary on Differences from Karpathy's Run

Architecture. Differential attention (Ye et al., ICLR 2025) vs Karpathy's standard MHA. Diff_attn suppresses noisy attention patterns and sharpens focus on relevant tokens, which may benefit multi-step arithmetic reasoning where attending to the right intermediate value matters.

SFT data. Our MetaMathQA + DART-Math augmentation gives RL a much stronger prior over GSM8K-style reasoning, explaining the higher absolute scores throughout.

Training speed. KV cache implemented for diff_attn restores O(T) generation. Remaining ~2.5×

slowdown vs Karpathy is from diff_attn's dual attention calls, not the cache.

Cost

Stage	GPU	Time	Cost
RL training (466 steps)	8×H100	~4 hrs	~$112
Eval + completion collection (combined)	8×H100	~60 min	~$28
Total		**~5 hrs**	**~$140**

Karpathy reported ~1.5 hrs training time in discussion #1 but did not publish the cost. Our run takes longer due to differential attention requiring 2× attention calls per forward pass, even with KV cache implemented. Karpathy's standard MHA has no such overhead.

Key Files

File	Role
`scripts/collect_completions.py`	DDP-parallel completion collection with accuracy summary
`runs/part3_rl_modal.py`	Modal pipeline: train → combined eval + collect completions
`dev/part3_analysis.py`	EDA: 4 plots + error categorization

dev/part3_completions.jsonl	1319 completions with correctness labels

The code changes for this part are available in the following branch:

https://github.com/jonathanjschen913/nanochat/tree/a4_part3

A pull request for these changes can also be viewed here:

https://github.com/jonathanjschen913/nanochat/pull/5

4 Part Four: Introducing a more complex rewards system

Part 3's error analysis revealed four failure modes in the binary-reward RL model:

Error Type	Count	% of Errors
Wrong setup / large error	767	58.2%
Arithmetic error (small)	107	8.1%
Unit / scale error	64	4.9%
Missing answer (#### absent)	44	3.3%

The binary reward (1.0 correct, 0.0 wrong) provides no gradient signal for partial progress — a response that reasons correctly but makes one arithmetic slip receives the same reward as one that misunderstands the problem entirely. We designed three additional reward functions targeting these failure modes, then combined them into a weighted ensemble.

Reward Function Definitions

All reward functions share the same interface: they receive the predicted numeric answer, the reference answer, and the full assistant response text.

1. Binary (Part 3 baseline)

```
reward_binary(pred, ref, response):
  return 1.0 if pred == ref else 0.0
```

No partial credit. This is the original reward from Part 3.

2. Format Reward

```
reward_format(pred, ref, response):
  if pred == ref: return 1.0
  if "####" in response: return 0.2
  return 0.0
```

3.3% of Part 3 errors had missing `####` markers — the model reasoned correctly but failed to produce a parseable answer. This reward gives 0.2 partial credit for including the answer marker even when the numeric answer is wrong, incentivizing the model to always produce structured output.

3. Tolerance Reward

```
reward_tolerance(pred, ref, response):
  if pred == ref: return 1.0
  if ref != 0 and |pred - ref| / |ref| <= 0.10: return 0.5
  return 0.0
```

8.1% of Part 3 errors were small arithmetic mistakes where the model's answer was close but not exact. This reward gives 0.5 partial credit for answers within 10% of the gold answer, providing gradient signal for near-misses rather than treating them identically to completely wrong answers. We also expected this to partially cover unit/scale errors (4.9%), since some unit confusions (e.g. forgetting to convert units in a final step) produce answers numerically close to the gold — and indeed, tolerance achieved the lowest unit/scale error rate in our results (1.1% vs 1.9% baseline).

4. Steps Reward

```
reward_steps(pred, ref, response):
  if pred == ref: return 1.0
  credit = 0.0
```

```
    if response has >= 3 non-empty lines: credit += 0.1
    if response contains >= 2 equals signs: credit += 0.1
    if response has >= 2 lines with digits: credit += 0.1
    return credit  # max 0.3
```

58.2% of Part 3 errors were wrong problem setup — the dominant failure mode. This reward gives partial credit (up to 0.3) for showing structured mathematical work: multi-line reasoning, equations, and intermediate calculations. The hypothesis is that rewarding the process of structured problem-solving, even when the final answer is wrong, will encourage the model to develop better reasoning chains.

5. Combined Reward

```
reward_combined(pred, ref, response):
    return 0.2 * format + 0.3 * tolerance + 0.5 * steps
```

A weighted ensemble giving the most weight to steps (targeting the dominant failure mode), moderate weight to tolerance (targeting arithmetic errors), and lower weight to format (targeting the least common error type).

Training Configuration

All runs use identical hyperparameters matching Part 3 / Karpathy's defaults, starting from the same `sft_combo` checkpoint (20% GSM8K before RL):

Parameter	Value
Starting checkpoint	sft_combo (Part 2 best)
Epochs	1 (467 steps)
Examples per step	16

Samples per example	16
Max new tokens	256
Temperature	1.0
Top-k	50
Device batch size	8
GPU	8x H100

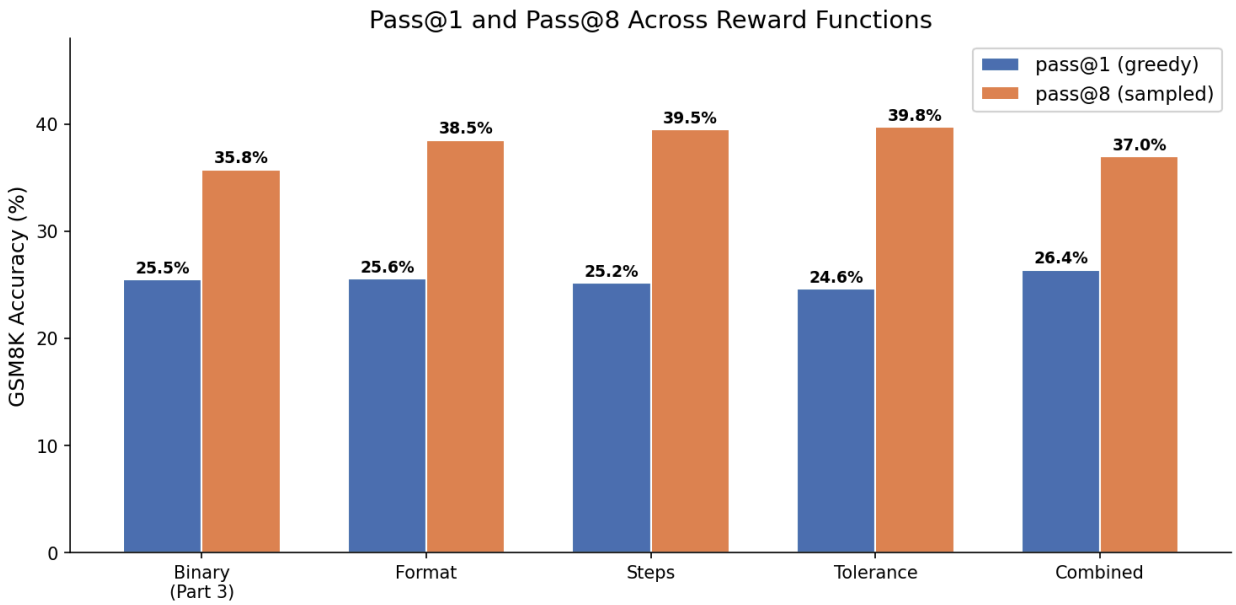
Results

Overall Accuracy

Run	Reward	pass@1 (greedy)	pass@8 (sampled)
Part 3 baseline	Binary	25.5% (337/1319)	35.75%
Format	Format	25.55% (337/1319)	38.50%
Steps	Steps	25.17% (332/1319)	39.50%

Tolerance	Tolerance	24.64% (325/1319)	39.75%
Combined	**0.2F + 0.3T + 0.5S**	**26.38% (348/1319)**	**37.00%**

Key findings: The combined reward achieves the highest pass@1 (26.38%), a +0.88pp improvement over the binary baseline. Individual rewards show marginal pass@1 differences (within ~1pp of baseline) but notably higher pass@8 scores (38.5-39.75% vs 35.75%), suggesting the shaped rewards improve the model's capability ceiling more than its greedy accuracy.



Error Category Breakdown

Category	Binary (Part 3)	Format	Steps	Tolerance	Combined
Correct	337 (25.5%)	337 (25.5%)	332 (25.2%)	325 (24.6%)	**348 (26.4%)**

Missing ####	44 (3.3%)	**10 (0.8%)**	21 (1.6%)	14 (1.1%)	16 (1.2%)
Unit/ Scale error	25 (1.9%)	21 (1.6%)	20 (1.5%)	**14 (1.1%)**	24 (1.8%)
Arithmetic	371 (28.1%)	374 (28.4%)	351 (26.6%)	358 (27.1%)	349 (26.5%)
Wrong setup/ large	542 (41.1%)	577 (43.7%)	595 (45.1%)	608 (46.1%)	582 (44.1%)

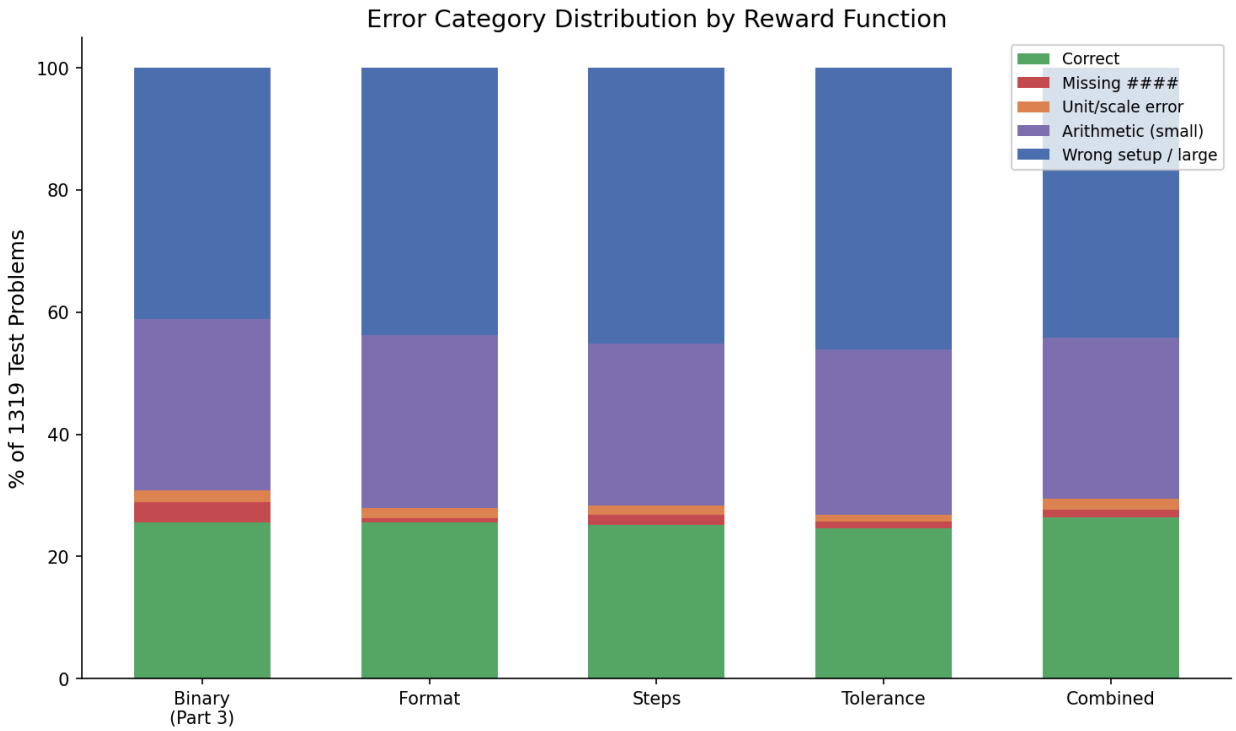
Observations

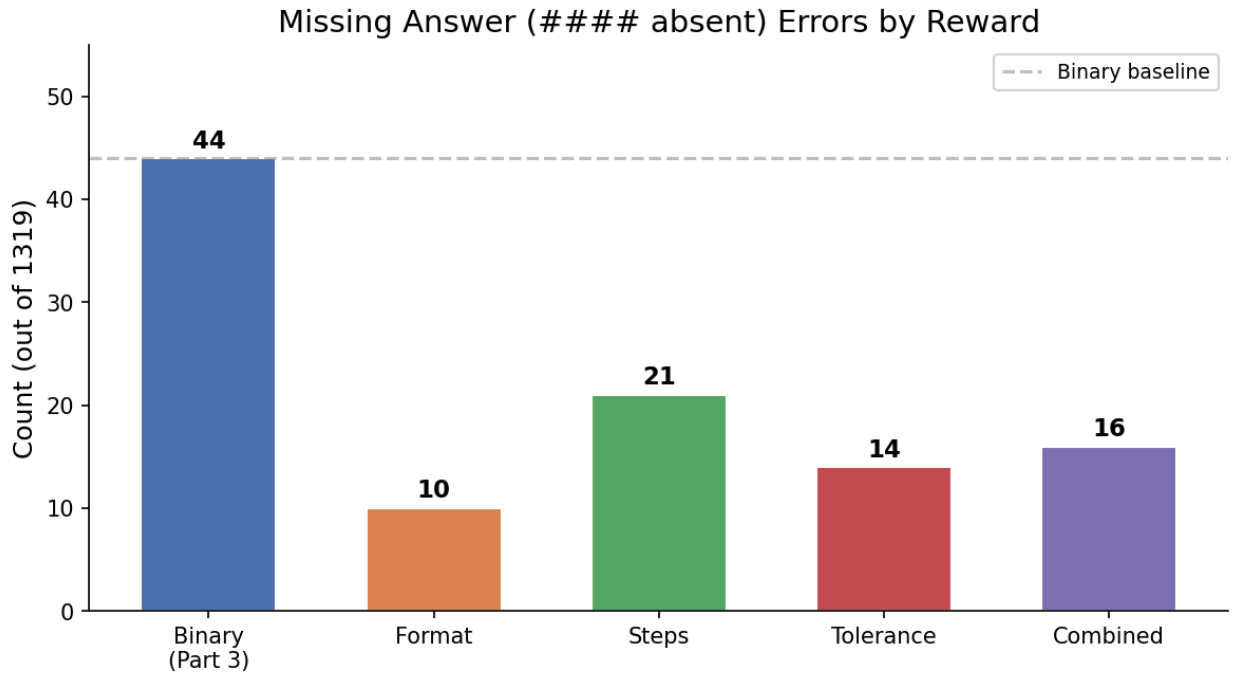
1. **Format reward eliminates missing-answer failures.** The binary baseline has 44 missing #### errors (3.3%); the format reward reduces this to just 10 (0.8%) — a 77% reduction. All Part 4 variants show improvement here, but the format reward is most effective since it directly rewards the marker.

2. **Tolerance reward reduces unit/scale errors.** Tolerance achieves the lowest unit/scale error rate (1.1% vs 1.9% baseline). By providing partial credit for numerically close answers, the model learns to avoid gross magnitude errors.

3. **Wrong setup remains dominant across all models.** All models show 41-46% wrong setup errors. The shaped rewards have not fundamentally improved problem comprehension — this likely requires architectural changes or better SFT data rather than reward shaping alone.

4. **Trade-off: partial-credit rewards shift errors between categories.** The individual shaped rewards (format, steps, tolerance) show slightly higher wrong-setup rates than the binary baseline (43-46% vs 41%). This suggests the partial credit incentivizes the model to attempt more problems (reducing missing answers) but doesn't always produce correct reasoning.



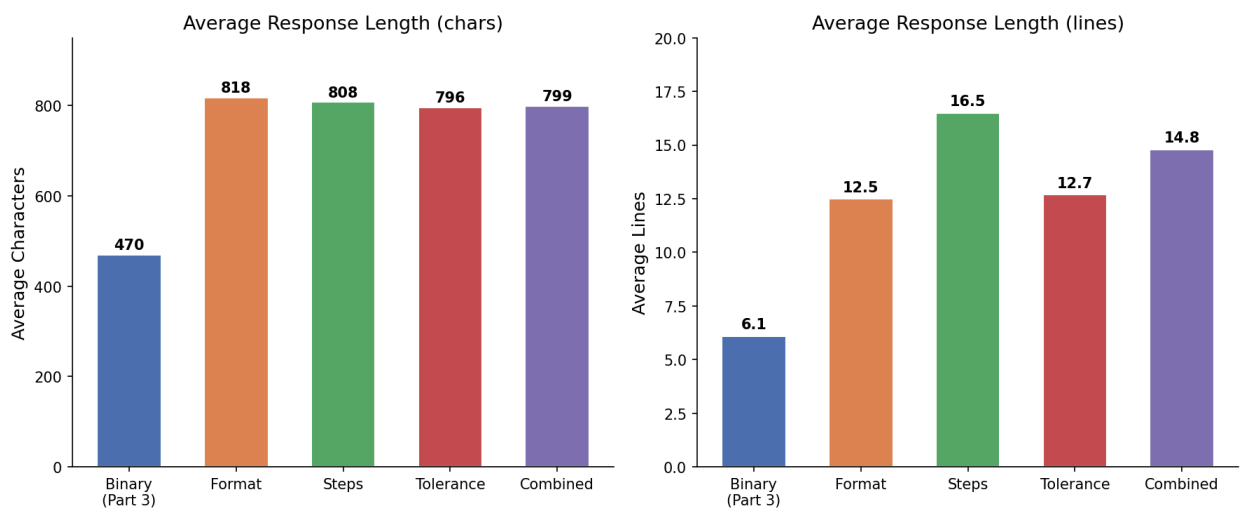


Response Length

Model	Avg chars	Avg lines

Binary (part 3)	470	6.1
Format	818	12.5
Steps	808	**16.5**
Tolerance	796	12.7
Combined	799	14.8

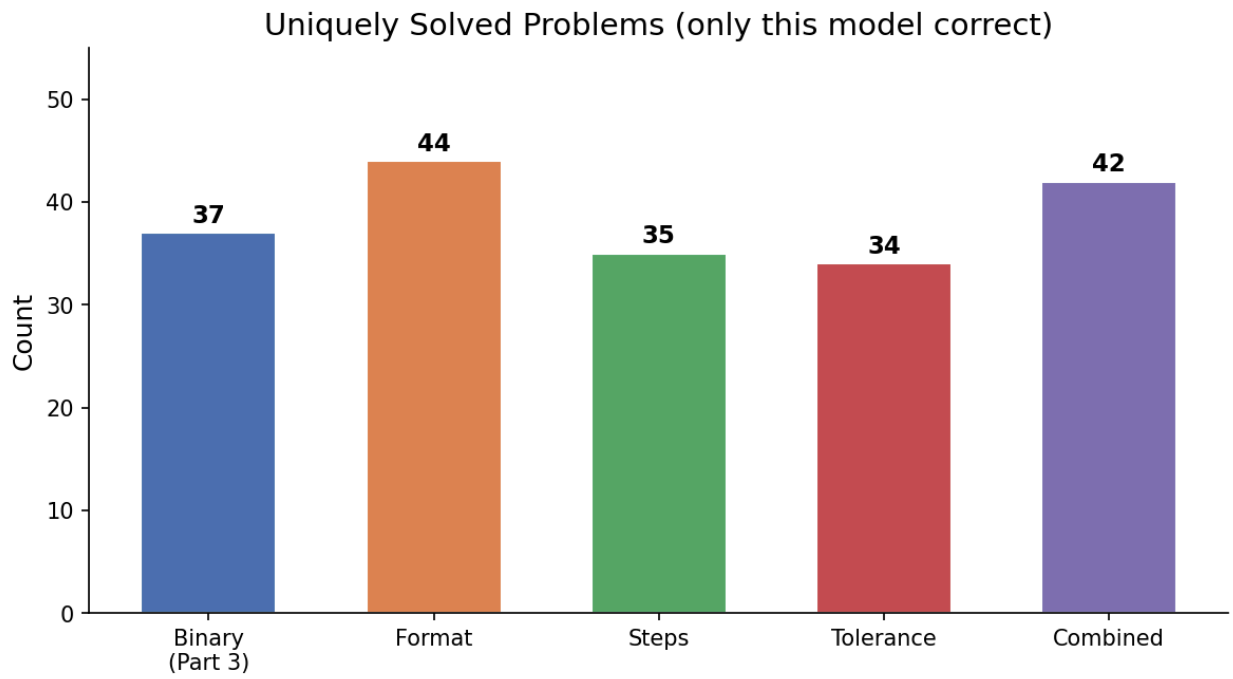
All Part 4 models produce ~70% longer responses than the binary baseline. The steps reward produces the most lines (16.5 avg), consistent with its design rewarding multi-line reasoning. The binary baseline is shortest — without incentive to elaborate, the model often produces minimal output.



Unique Problem Coverage

Model	Problems only this model solves
Binary (part 3)	37
Format	**44**
Steps	35
Tolerance	34
Combined	42

Despite similar aggregate accuracy, each reward function solves a different subset of problems. The format reward uniquely rescues 44 problems no other model solves — the highest count. This demonstrates that reward shaping genuinely diversifies model behavior rather than simply improving the same problems.



Illustrative Examples

Example A — Algebraic setup (board cutting)
> Q: "Ian has a 40-foot board. The longer piece is 4x the shorter. What is the longer piece?"
> Gold: 32

- Binary baseline: Divides 40/12 = 20, then 4x20 = 80. Wrong. Fails to set up `x + 4x = 40`.
- **Combined & Steps:** Both correctly set `x + 4x = 40 -> x=8 -> 4x=32`. **Correct.**
- The steps reward's incentive for showing multi-line work pushed the model to write out the algebraic setup, which this problem specifically required.

Example B — Age problem (requires constraint satisfaction)
> Q: "Jerry is twice as old as he was 5 years ago. How old will he be in 3 years?"
> Gold: 13

- Binary & Steps: Mangle the constraint, producing wrong setups.
- Tolerance & Format: Correctly reason 2x5=10, 10+3=13. Correct.
- The format reward's emphasis on structured output appears to cut off hallucinated reasoning loops before they corrupt the answer.

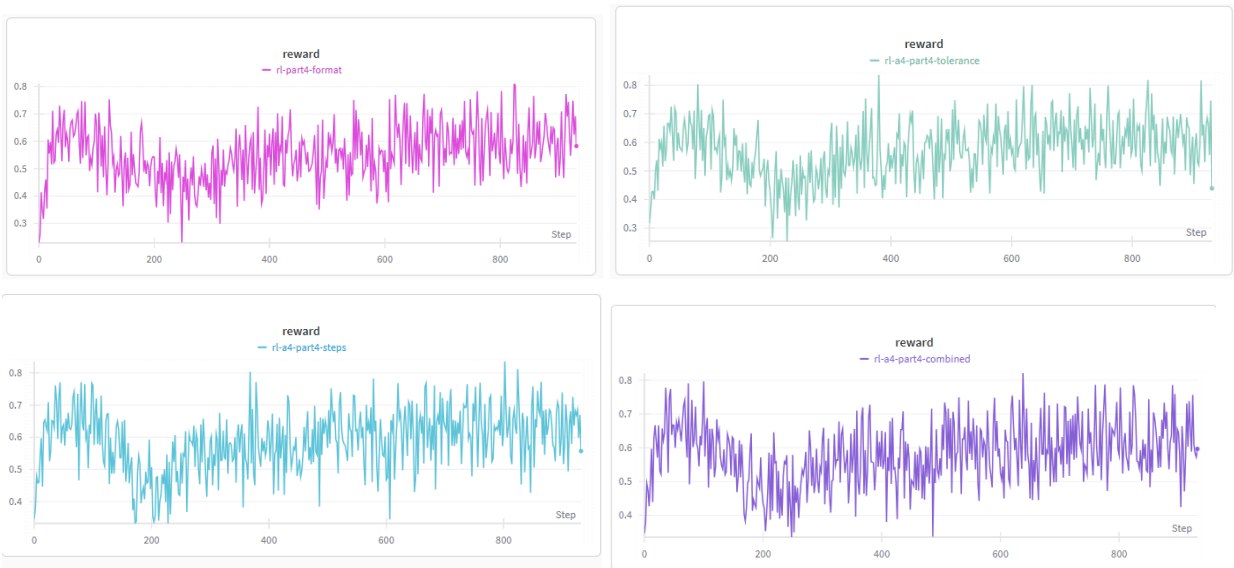
Example C — Pure arithmetic (stamp counting)
> Q: Counting stamps across categories, final sum required.
> Gold: 45

- All models except Tolerance: Set up the problem identically but compute the final sum incorrectly (53, 48, 41).

- Tolerance: Computes 16+19+10 = 45. Correct.
- This is a pure arithmetic case — all models understand the problem structure, but only the tolerance-trained model executes the final addition correctly. The partial credit for near-misses appears to train more careful arithmetic.

Reward Curves

Training reward curves for all five runs (binary baseline + four Part 4 variants), overlaid from W&B:



All runs show consistent reward improvement over 467 steps. Key observations from the curves:

- Steps achieves the highest final training reward (~0.75), closely followed by the binary baseline (~0.75). This is expected: the steps reward gives partial credit (up to 0.3) for showing work even on wrong answers, inflating the raw reward value.
- Tolerance has the lowest final training reward (~0.68). Its 0.5 partial-credit band is narrower than steps' 0.3 across three criteria, so fewer wrong answers earn any credit.
- Combined and Format land in the middle (~0.72–0.74), reflecting their blended or single-signal partial credit.
- Sequence length (W&B middle chart) confirms the response length analysis: format produces the longest sequences (~308 tokens), while the binary baseline produces the shortest (~294). All Part 4 variants produce longer outputs than the baseline, consistent with partial-credit incentives encouraging more detailed reasoning.

Importantly, a higher training reward does not directly correspond to higher pass@1 accuracy. The combined reward has the best pass@1 (26.38%) despite not having the highest training reward — this suggests the multi-signal reward provides a richer learning signal even though its absolute reward values are moderate.

Summary and Discussion

Metric	Binary (Part 3)	Format	Steps	Tolerance	Combined
pass@1 greedy	25.5%	25.55%	25.17%	24.64%	**26.38%**
pass@8 sampled	35.75%	38.50%	39.50%	**39.75%**	37.00%
Missing #### errors	44	10	21	**14**	16
Wrong setup errors	542	577	595	608	582

Metric	Binary (Part 3)	Format	Steps	Tolerance	Combined
Avg response length	470 chars	818 chars	808 chars	796 chars	799 chars
Unique problems solved	37	44	35	34	42

Impact of each reward:

- Format is the most targeted fix: it nearly eliminates the missing-answer failure mode (3.3% -> 0.8%) with no accuracy cost. It also uniquely solves the most problems (44).
- Steps produces the most verbose, structured responses (16.5 lines avg) and achieves the second-highest pass@8 (39.50%), suggesting it improves the model's reasoning capability even when greedy decoding doesn't capture it.
- Tolerance shows the highest pass@8 (39.75%) but lowest pass@1 (24.64%), indicating it may encourage the model to explore more diverse solutions — beneficial under sampling but not at greedy temperature.
- Combined achieves the best pass@1 by blending all three signals, confirming that the reward components are complementary rather than redundant.

Limitations: The dominant failure mode — wrong problem setup (41-46% across all models) — persists regardless of reward shaping. This suggests the model's fundamental reasoning capability is bounded by the SFT data and model scale, not by the RL reward signal. Further improvement likely requires better pretraining/SFT data for mathematical reasoning, or architectural changes that improve multi-step planning.

Cost

Run	GPU	Training Time	Eval + Completions	Cost
Combined	8×H100	~3.5 hrs	~1 hr	~$126
Format	8×H100	~4 hrs	~1 hr	~$140
Steps	8×H100	~4 hrs	~1 hr	~$140
Tolerance	8×H100	~4 hrs	~1 hr	~$140
Total		**~16 hrs**	**~4 hrs**	**~$546**

Key Files

File	Role
`tasks/gsm8k.py`	Reward function definitions (binary, format, tolerance, steps, combined)
`scripts/chat_rl.py`	RL training — --reward-fn and --output-tag flags added
`runs/part4_rl_modal.py`	Modal pipeline: single server-side function (train -> collect -> eval)

`dev/part4_completions_*.jsonl`	1319 completions per model for error analysis
`dev/part3_completions.jsonl`	Binary baseline completions (Part 3)

The code changes for this part are available in the following branch:

https://github.com/jonathanjschen913/nanochat/tree/a4_part4

A pull request for these changes can also be viewed here:

https://github.com/jonathanjschen913/nanochat/pull/6

References

Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805.

Karpathy, A. (2025). nanochat: A tiny chatbot arena and training harness. https://github.com/karpathy/nanochat/discussions/481

Cobbe, K., Kosaraju, V., Bavarian, M., et al. (2021). Training Verifiers to Solve Math Word Problems. arXiv preprint arXiv:2110.14168.

Shao, Z., Wang, P., Zhu, Q., et al. (2024). GRPO: Group Relative Policy Optimization for Language Model Alignment. arXiv preprint arXiv:2402.05191.

Karpathy, A. (2025). nanochat: A tiny chatbot arena and training harness. https://github.com/karpathy/nanochat/discussions/1

Ye, T., Dong, L., Xia, Y., et al. (2025). Differential Attention. ICLR 2025. arXiv:2410.05258.

Yu, J., et al. (2024). DAPO: An Open-Source LLM Reinforcement Learning System at Scale. arXiv preprint arXiv:2503.14476.